PRÁCTICA PRIMERA

PARSER DESCENDENTE RECURSIVO

PROCESADORES DEL LENGUAJE
GRADO EN INGENIERÍA INFORMÁTICA
MARZO 2026

Grupo 607

Maria Romero Martin:100472330@alumnos.uc3m.es
Guillermo García Ortega: 100472109@alumnos.uc3m.es

Tabla de Contenidos

1. INTRODUCCIÓN **2**
2. ESTUDIO DEL CÓDIGO BASE **2**
3. DISEÑO DE LA GRAMÁTICA **2**
3.1 FASE I: DISEÑO DE LA GRAMÁTICA INICIAL 3
3.2 FASE II: AMPLIACIÓN DE LA GRAMÁTICA 4
3.3 NIVELES SINTÁCTICO Y LÉXICO 6
4. DIAGRAMAS SINTÁCTICOS DE LA GRAMÁTICA **11**
5. PRUEBAS DE EVALUACIÓN Y TRADUCCIÓN **12**

1. INTRODUCCIÓN

El presente documento expone el desarrollo y las soluciones elaboradas correspondientes a la primera fase de la práctica de la asignatura de '*Procesadores del Lenguaje*'. En esta etapa inicial del proyecto se han diseñado y optimizado varias versiones de distintas gramáticas, con el objetivo de lograr la correcta implementación de un analizador sintáctico descendente recursivo.

El propósito principal de esta primera fase consiste en la construcción de un traductor capaz de transformar expresiones escritas en un lenguaje sencillo utilizando notación prefija en su representación equivalente en notación infija. Para ello, se han definido las reglas gramaticales pertinentes, se han analizado las propiedad sintácticas del lenguaje a considerar y se han desarrollado todos los mecanismos necesarios para garantizar un proceso de análisis y de traducción correcto.

2. ESTUDIO DEL CÓDIGO BASE

El primer paso para la implementación del parser descendente recursivo consiste en comprender y examinar el código base proporcionado. Este código incluye una estructura preliminar del analizador sintáctico donde se definen algunas de las funciones principales encargadas de inicializar el proceso de análisis, como por ejemplo las funciones **ParseAxiom()** y **ParseYourGrammar**(), aunque esta última función en el estado inicial aún no está implementada. En su estado actual, el programa únicamente valida correctamente las entradas que finalizan con un salto de línea **'\n'**.

Por otra parte, el código incorpora un analizador léxico con la función **rd_lex(),** responsable de leer la entrada u reconocer los distintos tokens. Concretamente, se definen los analizadores léxicos **T_OPERATOR**, **T_NUMBER** y **T_VARIBALE,** correspondientes respectivamente a operadores aritméticos, números y variables. Además, para gestionar esta información, se define la estructura **s_tokens**, que almacena los datos asociados al token actual y al que se ha leído previamente, facilitando así el análisis sintáctico.

3. DISEÑO DE LA GRAMÁTICA

Tras el estudio del código base, se ha abordado el diseño de la gramática del parser. Esta es la encargada de describir la sintaxis de las expresiones aritméticas escritas en notación prefija. Para facilitar su implementación, buscamos que la gramática fuera clara, estuviera bien estructurada y fuera lo suficientemente concisa, sin comprometer su capacidad de analizar correctamente las expresiones del lenguaje.

El proceso de desarrollo se llevó a cabo en dos fases. En la primera etapa se diseñó una versión simplificada de la gramática, orientada al reconocimiento de operaciones básicas formadas por un único operador y dos operandos numéricos de una sola cifra.

Posteriormente, a partir de nuestra primera base, se fueron refinando y ampliando las reglas de la gramática hasta que se obtuvo una solución que cumpliera con todos los objetivos establecidos.

3.1 FASE I: DISEÑO DE LA GRAMÁTICA INICIAL

A continuación, bajo estas líneas se expone nuestra primera versión de la gramática, donde sólo se reconocen operaciones básicas formadas por un operador y dos operandos numéricos de una sola cifra.

```
None
S ::= EXPRESION
EXPRESION ::= (OPERADOR PARAMETRO PARAMETRO)
OPERADOR ::= T_OPERATOR
PARAMETRO ::= NUMERO
NUMERO ::= T_NUMBER
```

Nótese que en esta gramática, **T_NUMBER** y **T_OPERATOR** son los tokens correspondientes a un número (0..9) y a un operador aritmético (+, -, *, /). Nótese también que la gramática en su estado actual tampoco reconoce términos como saltos de línea o espacios ya que estas consideraciones pasan a ser tratados por la función **rd_lex()** ya que pertenecen al nivel léxico.

Para que la gramática anterior pueda ser procesada por la herramienta JFLAP, se ha tenido que modificar la nomenclatura de los no terminales de la gramática para poder validarla como una gramática de tipo LL(1). Tras estas modificaciones, la gramática queda de la siguiente manera:

```
None
S ::= E
EXPRESION ::= (OPP)
OPERADOR ::= +
PARAMETRO ::= NUMERO
NUMERO ::= 1
```

En ella, se ha eliminado cualquier espacio posible y se han sustituido los tokens por valores concretos (‘1’ y el operador aritmético ‘+’) con el fin de facilitar la validación de la gramática.

Al introducir la gramática modificada en JFLAP obtenemos los siguientes resultados:

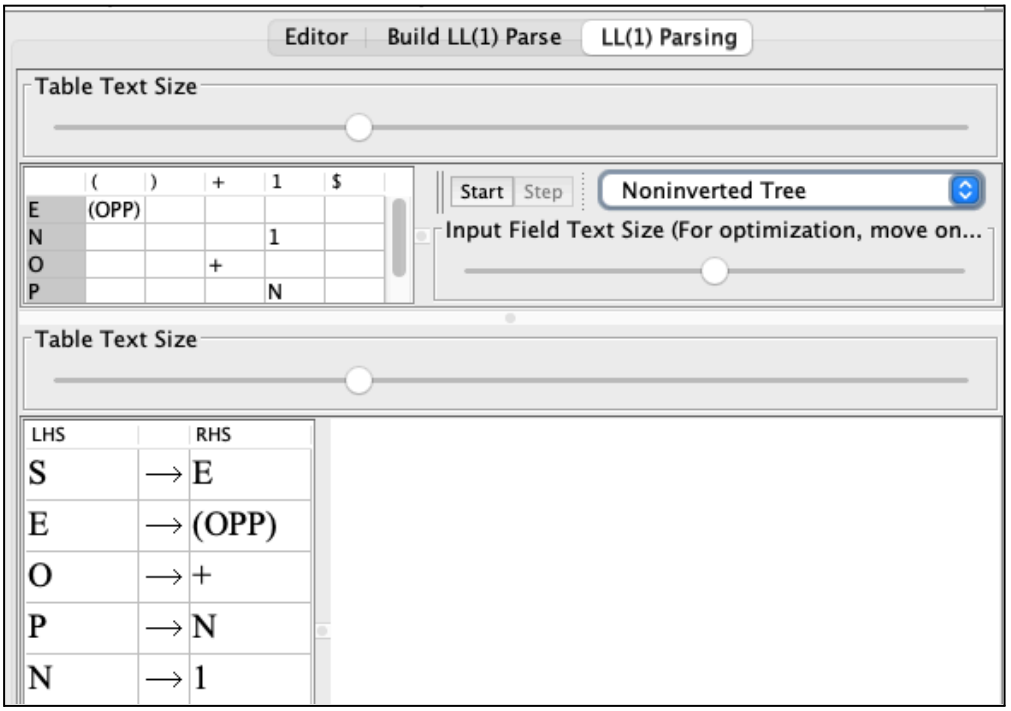


Figura 1. Tabla de parsing LL(1) generada en JFLAP a partir de la gramática básica definida para el reconocimiento de expresiones aritméticas.

Tal y como se puede observar, esta primera gramática cumple con los requisitos mínimos, pero para poder llegar al reconocimiento de variables, realizar asignaciones o llevar a cabo otras operaciones anidadas es imperativo añadir más reglas.

3.2 FASE II: AMPLIACIÓN DE LA GRAMÁTICA

La primera versión de nuestra gramática final se expone a continuación:

```
None
S ::= EXPRESION
EXPRESION ::= (OPERADOR PARAMETRO PARAMETRO) | (= VARIABLE PARAMETRO)
              | (? PARAMETRO PARAMETRO PARAMETRO)
              | ( = VARIABLE PARAMETRO PARAMETRO PARAMETRO)
PARAMETRO ::= EXPRESION | NUMERO | VARIABLE
OPERADOR ::= T_OPERATOR
NUMERO ::= T_NUMBER
VARIABLE ::= T_VARIABLE
```

El problema es que, a pesar de que esta gramática ahora cumple con las especificaciones requeridas, presenta un claro conflicto de tipo primero - primero en las reglas derivadas de

EXPRESION. Es por esto que resulta necesario factorizar dicha regla para poder eliminar el conflicto.

En primer lugar, creamos la siguiente iteración de una posible solución que eliminase los conflictos primero-primero derivados del uso de paréntesis en la regla productora de expresión. Tras ello, nuestra gramática quedó de la siguiente manera:

```
None
S ::= EXPRESION
EXPRESION ::= '(' CONTENIDO ')'
CONTENIDO ::= OPERADOR PARAMETRO PARAMETRO
              | '?' PARAMETRO PARAMETRO PARAMETRO
              | '=' VARIABLE ASIGNACION
ASIGNACION ::= PARAMETRO
              | PARAMETRO PARAMETRO PARAMETRO
PARAMETRO ::= EXPRESION | NUMERO | VARIABLE
OPERADOR ::= T_OPERATOR
NUMERO ::= T_NUMBER
VARIABLE ::= T_VARIABLE
```

No obstante, esta aproximación aún generaba posibles conflictos primero-primero en la regla productora **ASIGNACION**, por lo que fue necesaria una segunda iteración para factorizar esta otra regla ya que ambas producciones empezaban por **PARAMETRO**. De esta manera, obtuvimos la siguiente gramática como versión final:

```
None
S ::= PARAMETRO
EXPRESION ::= '(' CONTENIDO ')'
CONTENIDO ::= OPERADOR PARAMETRO PARAMETRO
              | '?' PARAMETRO PARAMETRO PARAMETRO
              | '=' VARIABLE ASIGNACION
ASIGNACION ::= PARAMETRO ASIGNACION'
ASIGNACION' ::= λ
              | PARAMETRO PARAMETRO
PARAMETRO ::= EXPRESION | NUMERO | VARIABLE
OPERADOR ::= T_OPERATOR
NUMERO ::= T_NUMBER
VARIABLE ::= T_VARIABLE
```

Cabe destacar que para permitir la aceptación de entradas atómicas, se ha modificado también la producción del axioma. Ajustada a un formato que permita introducirse en JFLAP, esta gramática se compondría de la siguiente manera: (Véase siguiente página).

Estos son los conjuntos primero y siguiente que se utilizarán para la composición del parser dado lo obtenido en JFLAP para la gramática anterior. Cabe destacar que el no terminal **ASIGNACION'** ha sido sustituido por **X** para facilitar la introducción de la gramática en JFLAP.

	FIRST	FOLLOW
A	{ 1, a, (}	{) }
C	{ +, =, ? }	{) }
E	{ (}	{ 1, a, $, (,) }
N	{ 1 }	{ 1, a, $, (,) }
O	{ + }	{ 1, a, (}
P	{ 1, a, (}	{ 1, a, $, (,) }
S	{ 1, a, (}	{ $ }
V	{ a }	{ 1, a, $, (,) }
X	{ λ, 1, a, (}	{) }

	(	)	+	1	=	?	a	$
A	PX			PX			PX	
C			OPP		=VA	?PPP		
E	(C)							
N				1				
O			+					
P	E			N			V	
S	P			P			P	
V							a	
X	PP	λ		PP			PP	

Figuras 2 y 3. Conjuntos FIRST y FOLLOW generados para la gramática final y Tabla de parsing LL(1) generada en JFLAP

3.3 NIVELES SINTÁCTICO Y LÉXICO

Nivel Léxico:

Es el nivel encargado de identificar y clasificar los tokens válidos presentes en la entrada del programa. A partir de las especificaciones del enunciado, se pueden definir los distintos tipos de tokens:

TIPO DE TOKEN	DESCRIPCIÓN
T_NUMBER	Un número entero formado por uno o más dígitos. [0-9]+
T_VARIABLE	Una variable formada por una letra seguida opcionalmente de otra letra o dígito. [a-zA-Z]([a-zA-Z0-9])?
T_OPERATOR	Operadores aritméticos. [+\-*/]
ASSIGN	Símbolo '=' utilizado para asignaciones
TERNARY	El símbolo '?' utilizado para el operador condicional
LPAREN	El símbolo '(' que indica el inicio de una expresión
RPAREN	El símbolo ')' que indica el final de una expresión

Además, el analizador léxico ignora cualquier espacio o salto de línea, por lo que solo se procesan los tokens relevantes para el análisis sintáctico.

Nivel Sintáctico

El analizador sintáctico tiene como objetivo comprobar que una secuencia de tokens que se obtiene durante la lectura del análisis léxico respeta las reglas de formación que se han establecido en la última versión de nuestra gramática, (la versión final de la Fase II)

3. DISEÑO DEL PARSER

Para el correcto desarrollo del parser es necesario tener en cuenta dos aspectos principales: el reconocimiento y el almacenamiento de tokens para la posterior traducción de notación infija a prefija. Para poder identificar los elementos de la entrada de la variable, se han utilizado los tokens proporcionados en el código base junto con la función **MatchSymbol()**. La traducción de notación infija a prefija se realiza dentro de las funciones del parser. Cada vez que se reconoce una de las producciones de la gramática, se genera una salida correspondiente, obteniendo de esta manera su equivalente en notación prefija. A continuación se exponen las funciones para esta gramática, empezando por el axioma.

C/C++

```
//Parse Axiom Function.
void ParseAxiom() // Axiom ::= P
{
   ParseP(); // Parse P
   printf("\n");
   if (tokens.token == '\n') {
       MatchSymbol ('\n');
   } else {
       rd_syntax_error ('\n', tokens.token, "-- Unexpected Token (Expected:%d=None,
Read:%d) at end of Parsing\n") ;
   }
}
```

La función **ParseP()** implementa la producción P ::= E | V | N, determinando el tipo de parámetro según el token actual y delegando el análisis en la función correspondiente.

C/C++

```
//Parse Parameter Function.
void ParseP() // P ::= E | V | N
{
   if (tokens.token == '(') {
       ParseE(); // Parse E
   } else if (tokens.token == T_VARIABLE) {
       ParseV(); // Parse V
   } else if (tokens.token == T_NUMBER) {
       ParseN(); // Parse N
   } else {
       rd_syntax_error (T_VARIABLE, tokens.token, "-- Unexpected Token
(Expected:%d=None, Read:%d) at end of Parsing\n") ;
   }
}
```

```C/C++
//Parse Expression Function.
void ParseE() // E ::= (C)
{
   if (tokens.token == '(') {
       MatchSymbol ('(') ;
       ParseC(); // Parse C
       if (tokens.token == ')') {
           MatchSymbol (')') ;
       } else {
           rd_syntax_error (')', tokens.token, "-- Unexpected Token (Expected:%d=None, Read:%d) at end of Parsing\n") ;
       }
   } else {
       rd_syntax_error ('(', tokens.token, "-- Unexpected Token (Expected:%d=None, Read:%d) at end of Parsing\n") ;
   }
}
```

La función **ParseE()** reconoce la producción `E ::= (C)`, verificando la presencia de los paréntesis que delimitan la expresión y delegando el análisis de su contenido en **ParseC()**. Asimismo, la función **ParseC()** implementa la producción `C ::= OPP | ?PPP | =VA`, determinando la estructura de la expresión según el token inicial (operador, ? o =) y generando durante el análisis la traducción correspondiente a notación infija.

```C/C++
//Parse Container Expression.
void ParseC() // C ::= OPP | ?PPP | =VA
{
   if (tokens.token == T_OPERATOR) {
       int op = tokens.token_val;
       ParseO(); // Parse O
       printf("(");
       ParseP(); // Parse first P
       printf(" %c ", op);
       ParseP(); // Parse second P
       printf(")");
   } else if (tokens.token == '?') {
       MatchSymbol ('?') ;
       printf("(");
       ParseP(); // condition
```

```
        printf(" ? ");
        ParseP(); // true branch
        printf(" : ");
        ParseP(); // false branch
        printf(")");
    } else if (tokens.token == '=') {
        MatchSymbol ('=') ;
        printf("(");
        ParseV(); // Parse V
        printf(" = ");
        ParseA(); // Parse A
        printf(")");
    } else {
        rd_syntax_error (T_OPERATOR, tokens.token, "-- Unexpected Token
(Expected:%d=None, Read:%d) at end of Parsing\n") ;
    }
}
```

La función **ParseX()** implementa la producción X ::= λ | PP, determinando si la asignación es simple o condicional según el token siguiente, y analizando dos parámetros adicionales cuando corresponda.

C/C++

```
//Function that Parses Prime Assignation.
void ParseA() // A ::= PX
{
    ParseP(); // Parse P
    ParseX(); // Parse X
}
//Function that Parses Assignation' (Represented as X in the code grammar)
void ParseX() // X ::= lambda | PP
{
    // lambda when we reach a point where there is nothing more to parse
    if (tokens.token == '\n' || tokens.token == ')') {
        return; // lambda case, do nothing
    } else {
        ParseP(); // Parse P
        ParseP(); // Parse P
    }
}
```

Las funciones **ParseO(), ParseN() y ParseV()** se encargan de reconocer operadores, números y variables respectivamente. Cada una verifica que el token actual corresponde al tipo esperado, genera su representación en la salida cuando procede y avanza al siguiente token mediante **MatchSymbol()**, produciendo un error sintáctico en caso contrario.

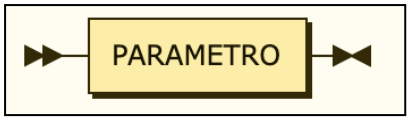
C/C++

```
//Function that Parses Operators.
int ParseO() //O ::= T_OPERATOR
{
   if (tokens.token == T_OPERATOR) {
       MatchSymbol (T_OPERATOR) ;
       return tokens.token_val;
   } else {
       rd_syntax_error (T_OPERATOR, tokens.token, "-- Unexpected Token
(Expected:%d=None, Read:%d) at end of Parsing\n") ;
       return -1;
   }
}
//Function that Parses Numbers.
void ParseN() // N ::= T_NUMBER
{
   if (tokens.token == T_NUMBER) {
       printf("%d", tokens.number);
       MatchSymbol (T_NUMBER) ;
   } else {
       rd_syntax_error (T_NUMBER, tokens.token, "-- Unexpected Token
(Expected:%d=None, Read:%d) at end of Parsing\n") ;
   }
}
//Function that Parses Variables.
void ParseV() // V ::= T_VARIABLE
{
   if (tokens.token == T_VARIABLE) {
       printf("%s", tokens.variable_name);
       MatchSymbol (T_VARIABLE) ;
   } else {
       rd_syntax_error (T_VARIABLE, tokens.token, "-- Unexpected Token
(Expected:%d=None, Read:%d) at end of Parsing\n") ;
   }
}
```

4. DIAGRAMAS SINTÁCTICOS DE LA GRAMÁTICA

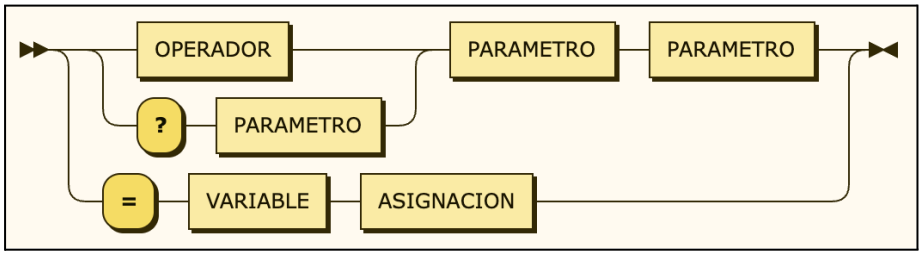
Para facilitar la comprensión del funcionamiento de la gramática, se ha utilizado la herramienta '*Railroad Diagram Generator*' para generar diagramas sintácticos de los símbolos no terminales. Estos diagramas a continuación muestran las posibles combinaciones que forman las secuencias válidas según nuestra gramática.

Para el AXIOMA (S)

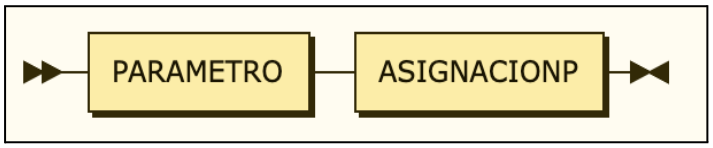


Para EXPRESION (E):

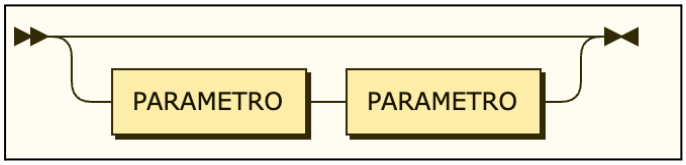
Para CONTENIDO (C):



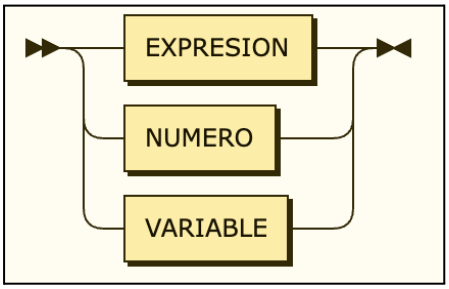
Para ASIGNACION (A):



Para ASIGNACIONP (X):



Para PARAMETRO (P):



5. PRUEBAS DE EVALUACIÓN Y TRADUCCIÓN

Para verificar el correcto funcionamiento del parser, hemos realizado una batería de pruebas utilizando un conjunto amplio de expresiones de entrada. En total, se han ejecutado aproximadamente 50 casos de prueba, lo que consideramos una cantidad adecuada para evaluar de forma completa el comportamiento del sistema.

#	Expresión de Entrada al Traductor	Resultado Esperado	Resultado Obtenido
1	A	A	A
2	321	321	321
3	z7	z7	z7
4	(+ A 122)	(A + 122)	(A + 122)
5	(= A (+ A 1))	(A = (A + 1))	(A = (A + 1))
6	(+ A (* B 4))	(A + (B * 4))	(A + (B * 4))
7	(= A (* 1234 B))	(A = (1234 * B))	(A = (1234 * B))
8	(= a (= b (= c3 (+ 2 3))))	(a = (b = (c3 = (2 + 3))))	(a = (b = (c3 = (2 + 3))))
9	(= a (+ (= b 2) (= c 3)))	(a = ((b = 2) + (c = 3)))	(a = ((b = 2) + (c = 3)))
10	(= az (= b (= c3 (+ 2 3))))	(az = (b = (c3 = (2 + 3))))	(az = (b = (c3 = (2 + 3))))
11	(= (+b 1) (+ a 1) (? a b c))	ERROR	ERROR
12	(= a1 (? (= b2 3) c4 d5))	(a1 = ((b2 = 3) ? c4 : d5))	(a1 = ((b2 = 3) ? c4 : d5))
13	(? (* A B) (+ C D) (/ E F))	((A * B) ? (C + D) : (E / F))	((A * B) ? (C + D) : (E / F))
14	(* (* A B) (* C D))	((A * B) * (C * D))	((A * B) * (C * D))
15	(+ (+ (+ A 1) 2) 3)	(((A + 1) + 2) + 3)	(((A + 1) + 2) + 3)
16	(= az (? (+ b1 2) (* c3 4) d5))	(az = ((b1 + 2) ? (c3 * 4) : d5))	(az = ((b1 + 2) ? (c3 * 4) : d5))
17	(= z7 (+ A1 (* B2 3)))	(z7 = (A1 + (B2 * 3)))	(z7 = (A1 + (B2 * 3)))
18	(+ (? A B C) 1)	((A ? B : C) + 1)	((A ? B : C) + 1)
19	(= A (+ B 1) (* C 2) (? D E F))	(A = ((B + 1) ? (C * 2) : (D ? E : F)))	(A = (B + 1)(C * 2)(D ? E : F))
20	(? (+ A 1) (* B 2) (- C 3))	((A + 1) ? (B * 2) : (C - 3))	((A + 1) ? (B * 2) : (C - 3))
21	(? A B (? C D E))	(A ? B : (C ? D : E))	(A ? B : (C ? D : E))

#	Expresión de Entrada al Traductor	Resultado Esperado	Resultado Obtenido
22	(? (? A B C) D E)	(A ? B : C) ? D : E)	((A ? B : C) ? D : E)
23	(= a (= b (= c 3)))	(a = (b = (c = 3)))	(a = (b = (c = 3)))
24	(+ (= a 2) 5)	((a = 2) + 5)	((a = 2) + 5)
25	(* (= x 4) (= y 7))	((x = 4) * (y = 7))	((x = 4) * (y = 7))
26	(? A B C)	(A ? B : C)	(A ? B : C)
27	(= y9 123)	(y9 = 123)	(y9 = 123)
28	(= X 7)	(X = 7)	(X = 7)
29	(* (+ A 1) (- B 2))	((A + 1) * (B - 2))	((A + 1) * (B - 2))
30	(+ (+ A 1) 2)	((A + 1) + 2)	((A + 1) + 2)
31	(+ A1 B2)	(A1 + B2)	(A1 + B2)
32	(/ 100 B)	(100 / B)	(100 / B)
33	(* z7 12)	(z7 * 12)	(z7 * 12)
34	99	99	99
35	B	B	B
36	(* z7 13)	(z7 * 13)	(z7 * 13)
37	(* e5 f6)	(e5 * f6)	(e5 * f6)
38	(/ g7 h8)	(g7 / h8)	(g7 / h8)
39	(/ 100 (/ 20 5))	(100 / (20 / 5))	(100 / (20 / 5))
40	(+ (* A B) (/ C D))	((A * B) + (C / D))	((A * B) + (C / D))
41	(* (+ A 1) (- B 2))	((A + 1) * (B - 2))	((A + 1) * (B - 2))
42	(/ (+ x1 y2) (* z3 4))	((x1 + y2) / (z3 * 4))	((x1 + y2) / (z3 * 4))
43	(= B 999)	(B = 999)	(B = 999)
44	(= A 1)	(A = 1)	(A = 1)
45	(? (+ A B) (* C D) (/ E F))	((A + B) ? (C * D) : (E / F))	((A + B) ? (C * D) : (E / F))
46	(= X (? (= A 1) (= B 2) (= C 3)))	(X = ((A = 1) ? (B = 2) : (C = 3)))	(X = ((A = 1) ? (B = 2) : (C = 3)))
47	(? (+ A 1) B C)	((A + 1) ? B : C)	((A + 1) ? B : C)